

Report 2026.07.22

cc-switch provider:

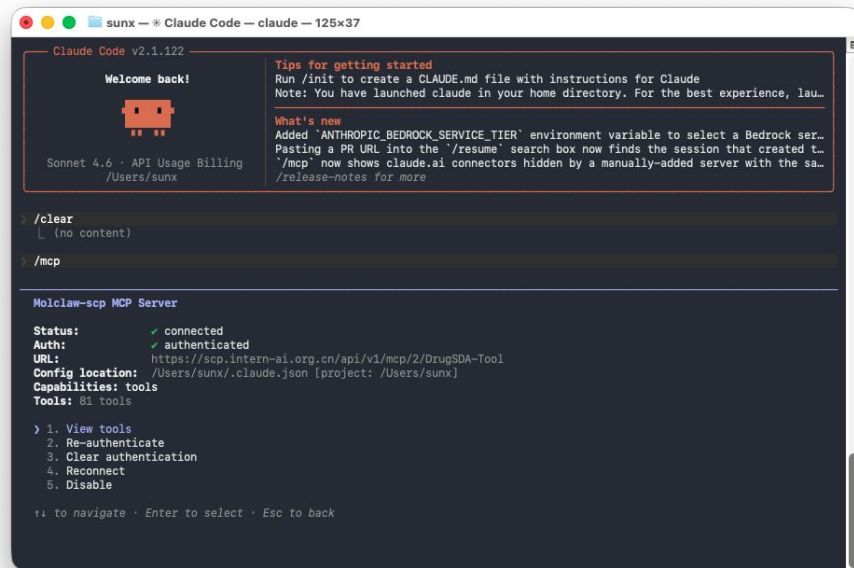
```
cc-switch --app claude provider add \  
--name "boyue-ds" \  
--base-url "http://35.220.164.252:3888/v1" \  
--api-key  
"sk-117W6Mf3WxL95hntXeeTNt2qqzEu8oVxI0AAJgOJwa6izT2n" \  
--model "deepseek-v4-pro"
```

mcp连接指令:

```
claude mcp add \  
molclaw-scp \  
--transport http \  
"https://scp.intern-ai.org.cn/api/v1/mcp/2/DrugSDA-Tool" \  
--header "SCP-HUB-API-KEY:  
sk-7e8704c0-b838-44e9-ab74-f116037bbf35"
```

claude

```
--dangerously-skip-permissions  
--verbose \  
--output-format  
stream-json -p "<prompt>" \  
complete_session.jsonl 2>&1
```



恢复 molclaw scp tool 的可用性, 由于新服务器迁移:

1. 新旧路径不一致, 代码里遗留旧路径
2. 环境问题, 如模型缓存丢失、程序缺失

仍未通过: 10 项

工具	当前确定原因
run_bioemu	本地没有完整模型缓存, 且服务器无法访问 Hugging Face
openawsem_sim	openawsem 环境缺少外部程序 stride
run_mmpbsa	当前 workdir 无法取得 Protein/配体组, 同时 GROMACS 运行环境也异常
gmx_mmpbsa_propro	gmx 缺少 libblas.so.3
prepare_complex	旧 add_hydrogen_plus.py 路径已修复; 现在阻塞于 libblas.so.3
prepare_protein_md	GROMACS 无法正常启动, 原因同上
karmadock_tool	KarmaDock 路径已修复; NumPy 导入时缺少 libcblas.so.3
chroma_monomer	chroma 环境无法导入 chroma 包
chroma_complex	同上
chroma_symmetry	同上

耗时长工具:

pred binding affinity boltz2

evobind tool

gmx mmpbsa workflow

gmx mmpbsa propro

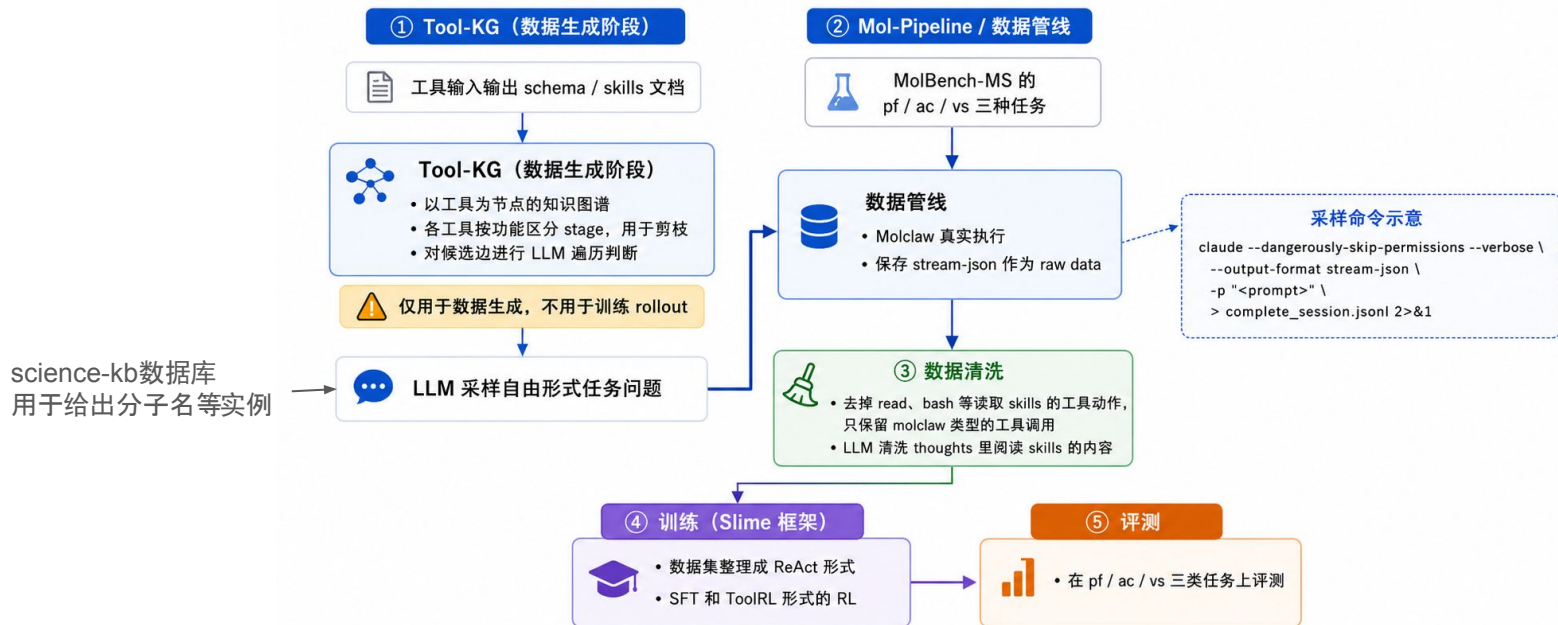
run openawsem simulation

prolif analyze pdb

同步做的还有: 维护 一键挂起服务.sh、测试工具可用性.py

由于本地的 mcp 地址不能公网开放, 只能用 scp.intern-ai.org.cn, 因此针对运行时间长的工具要改成异步

把整个项目 : *tool-kg* – *data-pipe* – *slime-wd* 迁移到了h集群机器上



关于从图上采样工具链的时候加入LLM推理节点:

- 在teacher model推理的时候自然地会在不同的工具调用之间出现thought
-

review了以后发现各个缓解都存在很严重的“职责重叠”/大量新旧代码逻辑不对齐的问题，有点屎山。

比如tool-kg里对边的判断应该全部交给LLM判断，但是存在python脚本里的类似功能，且会交给LLM，影响LLM的判断；

比如data-pipe里应该交给LLM完成的数据清洗工作，但是在python脚本清洗（主要是重构格式等功能）也出现类似功能；

所以花时间优化了tool-kg和data-pipe里存在的这种问题。

优化后运行了一个最小的流程（提取2个工具：`retrieve_protein_structure_by_gene_name`, `fix_pdb`；tool-kg 到 data-pipe 每一步一跑一个样本）

实例：

tool-kg正确判断出这两个边的联系为generates_full_input_for

```
{
  "schema_version": "tool_kg_graph_edge_v1",
  "edge_id": "pair::retrieve_protein_structure_by_gene_name__to__fix_pdb::edge::01",
  "pair_id": "pair::retrieve_protein_structure_by_gene_name__to__fix_pdb",
  "source_tool": "retrieve_protein_structure_by_gene_name",
  "target_tool": "fix_pdb",
  "source_stage": "target_structure_acquisition",
  "target_stage": "structure_rebuild_repair",
  "edge_type": "generates_full_input_for",
  "direct_transition": true,
  "relation_status": "valid",
  "confidence": 0.95,
  "source_slot": "output.prot_structure_path",
  "target_slot": "input.input_path",
  "evidence": ["snapshot::retrieve_protein_structure_by_gene_name", "snapshot::fix_pdb",
".claude/skills/L1_tools/molclaw-protein-structure-retrieve/SKILL.md", ".claude/skills/L1_tools/molclaw-fix-pdb/SKILL.md",
".claude/skills/L1_tools/molclaw-pdbfixer/SKILL.md", ".claude/skills/L2_workflows/01-target-protein-preparation.md"],
  "source_authority": "claude_adjudication",
  "eligible_for_sampling": true,
  "run_id": "minitest_20260720_133600"
}
```

根据这两个工具的链产生的问题：

```
{
  "status": "success",
  "public_question_text": "Retrieve the protein structure for the human ADRA2A gene (NCBI Taxonomy ID: 9606). Repair the PDB file to fix missing atoms and residues, add hydrogens, and remove heterogens. Return the path to the repaired structure for downstream analysis of [3H]rauwolscine binding (SMILES: COC(=O)[C@@H]1[C@@H](O)CC[C@H]2[C@@H]1C[C@@H]1N(C2)CCc2c1[nH]c1c2cccc1).",
  "question_payload": {
    "task": "retrieve_and_repair_protein_structure",
    "inputs": {
      "gene_name": "ADRA2A",
      "organism": "9606",
      "compound_name": "[3H]rauwolscine",
      "compound_smiles": "COC(=O)[C@@H]1[C@@H](O)CC[C@H]2[C@@H]1C[C@@H]1N(C2)CCc2c1[nH]c1c2cccc1"
    },
    "expected_output": "Path to a repaired PDB file of the ADRA2A protein structure with fixed missing atoms and residues, added hydrogens, and removed heterogens, suitable for downstream structural analysis or docking."
  },
  "rationale": "The grounding facts provide ADRA2A (gene name, UniProt P08913, 19 validated PDB entries) and the ligand [3H]rauwolscine with canonical SMILES. The toolchain blueprint pairs structure retrieval by gene name with PDBFixer repair. The organism taxonomy ID 9606 (Homo sapiens) follows directly from ADRA2A being a human gene. The compound and SMILES provide a realistic drug-discovery framing without being required for the structure-retrieval-and-repair execution path."
}
```

Teacher Model (MolClaw w/ deepseek-v4-pro) 运行轨迹经过清洗:

```
{
  "schema_version": "drug_agent_sft_react_json_v1",
  "id": "react_kg_dc982fb4cf810e83",
  "messages": [{
    "role": "system",
    "content": "You are a scientific agent operating under the canonical ReAct protocol.\nUse only real MolClaw calls from the recorded execution. Write scientific reasoning inside<thought>...</thought>, calls inside <tool_call>...</tool_call>, recorded results inside<observation>...</observation>, and the grounded result inside <final_answer>...</final_answer>.\nNever invent a tool result. Observations must come from execution history, and the final answer\nmust be supported by the task result and recorded observations.",
    "step_loss_mask": 0
  }, {
    "role": "user",
    "content": "Retrieve the protein structure for the human ADRA2A gene (NCBI Taxonomy ID: 9606). Repair the PDB file to fix missing atoms and residues, add hydrogens, and remove heterogens. Return the path to the repaired structure for downstream analysis of [3H]rauwolscine binding (SMILES: COC(=O)[C@@H]1[C@@H](O)CC[C@H]2[C@@H]1C[C@@H]1N(C2)CCc2c1[nH]c1c2cccc1).",
    "step_loss_mask": 0
  }, {
    "role": "assistant",
    "content": "\n\n<thought>The task requires two computational steps: retrieve the ADRA2A protein structure by gene name (organism 9606), then repair the PDB by adding hydrogens, removing heterogens, and fixing missing atoms and residues. The necessary MCP tools — retrieve_protein_structure_by_gene_name and fix_pdb — are available.</thought>\n\n<thought>Step 1: Retrieve the protein structure for ADRA2A.</thought>\n<thought>Retrieving the ADRA2A protein structure.</thought>\n<tool_call>{\n  \"arguments\": {\n    \"gene_name\": \"ADRA2A\", \"organism\": \"9606\", \"sort_by\": \"length\", \"tool_name\": \"retrieve_protein_structure_by_gene_name\"}\n}</tool_call>",
    "step_loss_mask": 1
  }, {
    "role": "user",
    "content": "<observation tool_name='retrieve_protein_structure_by_gene_name'>{\n  \"content\": {\n    \"msg\": \"成功检索到基因ADRA2A 的蛋白质结构\", \"prot_structure_path\": \"<artifact:structure/P08913_6K42.pdb>\", \"status\": \"success\", \"is_error\": false, \"status\": \"success\", \"tool_name\": \"retrieve_protein_structure_by_gene_name\"}\n}</observation>",
    "step_loss_mask": 0
  }, {
    "role": "assistant",
    "content": "<thought>The protein structure was retrieved successfully at <artifact:structure/P08913_6K42.pdb>. This is the ADRA2A structure (UniProt ID: P08913, PDB ID: 6K42). The next step is to repair it with fix_pdb: add hydrogens, remove heterogens, and fix missing atoms and residues.</thought>\n\n<thought>Structure retrieved: **P08913 (PDB: 6K42)** — the human ADRA2A adrenergic receptor. Now **Step 2:** Repair with 'fix_pdb' — fix missing atoms, add hydrogens, remove heterogens.</thought>\n\n<tool_call>{\n  \"arguments\": {\n    \"add_hydrogens\": true, \"add_missing_residues\": true, \"input_path\": \"<artifact:structure/P08913_6K42.pdb>\", \"ph\": 7, \"remove_heterogens\": true, \"remove_water\": true, \"tool_name\": \"fix_pdb\"}\n}</tool_call>",
    "step_loss_mask": 1
  }, {
    "role": "assistant",
    "content": "\n\n<thought>The protein structure was repaired successfully at <artifact:structure/P08913_6K42.pdb>. This is the ADRA2A structure (UniProt ID: P08913, PDB ID: 6K42). The next step is to return the path to the repaired structure for downstream analysis of [3H]rauwolscine binding.</thought>\n\n<final_answer>The path to the repaired structure for downstream analysis of [3H]rauwolscine binding is <artifact:structure/P08913_6K42.pdb>.\n\n</final_answer>"
  }
]
```


- 对已经跑出来的现有的molbench-ms分子筛选的三类任务样本, 如果经过现在的数据清洗流程, 会有很多样本被rejected
- 现在的数据清洗的规则过于严格, 需要再review这些规则后再改

允许在数据中让它读tool-level skills, 也允许它做写run log, result md等, 也就是说实际运行的时候要给我们训练的智能体有这些如read write edit等文件操作的工具的

Report 2026.07.29

工具侧的改进:

1. 解决超时问题 – 长时间工具每20s会返回still running的信息, 能够保护不会InternScience的服务端判定超时
2. 测试了占用GPU的工具的显存占用峰值, 并添加了自动检查、选用GPU的逻辑
3. 检索修复一些细节问题: 如 skills和实际工具存在部分不一致; retrieve工具默认走代理导致不能联网; 工具内部超时限制

Tool-KG生成的问题方面

生成了40道题，跑完以后让codex检查“题目意义、执行质量”

题目意义和执行质量是分开评价的。

题目意义主要看“这道题值不值得做”：

- 科学问题是否合理，工具链是否符合该方法学。
- 是否对应真实的药物发现决策，例如候选排序、机制比较、结构准备或风险筛选。
- 输入是否充分且可信，包括真实靶点、配体、结构和实验参照。
- 多个步骤是否形成连贯的科学链条，而不是为了串工具机械拼接。
- 任务复杂度是否与现有工具能力匹配。
- 结果是否能支持有意义的下一步行动。

执行质量主要看“这次实际上做得怎么样”：

- 核心要求是否真正完成，而不只是 `return_code=0`。
- 是否使用了题目指定的结构、方法、参数和对象。
- 工具调用和数值是否真实、有 observation 支持。
- 失败后的 fallback 是否科学合理，是否改变了原任务含义。
- 候选覆盖是否完整，例如要求 100 个却只评估 5 个不能算完成。
- docking pose、binding site、单位、排名和最终结论是否一致、可信。
- 必要 artifact 是否存在，可视化或结果文件是否真实生成。
- 是否明确报告限制，没有把未完成或替代结果包装成成功。

简单说：

text

题目意义 = 问题设计是否科学、有价值、可执行

执行质量 = 实际轨迹是否完整、忠实、可靠地解决了这个问题

题目意义评价

水平	数量	比例
高	21	52.5%
中高	8	20.0%
中	7	17.5%
中低	4	10.0%
低	0	0%
合计	40	100%

整体上，29/40（72.5%）题目的意义达到中高或高。说明 Tool-KG 採样的问题总体不是“没有科学价值”，主要问题出在执行能力与题目复杂度不匹配。

执行质量评价

水平	数量	比例
高	11	27.5%
中高	7	17.5%
中	10	25.0%
中低	6	15.0%
低	6	15.0%
合计	40	100%

执行质量达到中高或高的共有 18/40（45%）；中及以上共有 28/40（70%）。

6 条低质量和多数中低

1. 任务规模超过同步

- RIPK1: 101 个
- CMKLR1: 95
- HRH4: 73 个
- 大批量任务在

2. 工具适用域不匹配

- QuickVina 用于
- HDock 接收由
- FoldX monom
- Mol2Mol 模型
- OpenAWSEM

3. 核心工具仍有服务

- 同步 pred_bi
- KarmaDock 实
- FoldX BuildMo
- all-atom GRO

4. fallback 改变了任

- all-atom MD 被
- 指定实验结构制
- 含磷 reference
- ensemble inte

5. docking box 或 re

- HDAC6 dockin
- centroid box 结
- 未正确使用 co

6. 可视化基础设施不

- PyMOL 3D ren
- ligand mode 不
- 这类问题通常

```
{
  "schema_version": "drug_agent_sft_react_json_v1",
  "id": "react_ac_9ef21a7d1116cf38",
  "messages": [
    {
      "role": "system",
      "content": "You are a scientific agent operating under the canonical ReAct protocol.\nUse only recorded MolClaw calls and supported local file/skill calls from the recorded execution.\nWrite scientific reasoning inside<thought>...</thought>, calls inside <tool_call>...</tool_call>, recorded results inside<observation>...</observation>, and the grounded result inside <final_answer>...</final_answer>.\nNever invent a tool result. Observations must come from execution history, and the final answer\nmust be supported by the task result and recorded observations.",
      "step_loss_mask": 0
    },
    {
      "role": "user",
      "content": "You are a computational medicinal chemist. For the target Telomeric repeat-binding factor 2, you are given two small molecules:\n\nMolecule A: CC(=O)N[C@@H](CC(C)C)C(=O)N[C@@H](C)C(=O)N[C@@H](CC(C)C)C(=O)N[C@@H](CCCCN)C(=O)N[C@@H](Cc1ccc(O)cc1)C(=O)N[C@@H](CC(C)C)C(=O)N[C@@H](C(=O)N[C@@H](C(=O)N1CCC[C@@H]1C(=O)N[C@@H](C(N)=O)C(C)C)[C@@H](C)O)C(C)C\nMolecule B: CC(=O)N[C@@H](CC(C)C)C(=O)N[C@@H](C)C(=O)N[C@@H](CC(C)C)C(=O)N[C@@H](CCCCN)C(=O)N[C@@H](Cc1ccc(O)cc1)C(=O)N[C@@H](CC(C)C)C(=O)N[C@@H](C(=O)N[C@@H](C(=O)N1CCC[C@@H]1C(=O)N[C@@H](C(N)=O)C(C)C)[C@@H](C)O)C1CCCC1\n\nPredict which molecule is more likely to have higher binding affinity (lower Ki) against the target.\n\nUse available evidence and tool outputs to make a justified decision whenever possible.\n\nOnly output the corresponding SMILES.",
      "step_loss_mask": 0
    },
    {
      "role": "assistant",
      "content": "\n\n\n<thought>The task is to compare binding affinity of two peptide-like molecules against TERR2 (Telomeric repeat-binding factor 2). I'll use the Boltz-2 affinity prediction tool which can predict protein-ligand binding affinity.\n\nFirst, let me validate the SMILES and get the protein sequence/structure for TERR2.</thought>\n\n<tool_call>{\n  \"arguments\": {\n    \"smiles_list\": [\"CC(=O)N[C@@H](CC(C)C)C(=O)N[C@@H](C)C(=O)N[C@@H](CC(C)C)C(=O)N[C@@H](CCCCN)C(=O)N[C@@H](Cc1ccc(O)cc1)C(=O)N[C@@H](CC(C)C)C(=O)N[C@@H](C(=O)N[C@@H](C(=O)N1CCC[C@@H]1C(=O)N[C@@H](C(N)=O)C(C)C)[C@@H](C)O)C(C)C\", \"CC(=O)N[C@@H](CC(C)C)C(=O)N[C@@H](C)C(=O)N[C@@H](CC(C)C)C(=O)N[C@@H](CCCCN)C(=O)N[C@@H](Cc1ccc(O)cc1)C(=O)N[C@@H](CC(C)C)C(=O)N[C@@H](C(=O)N[C@@H](C(=O)N1CCC[C@@H]1C(=O)N[C@@H](C(N)=O)C(C)C)[C@@H](C)O)C1CCCC1\"]\n  },\n  \"tool_name\": \"is_valid_smile_s\"\n}</tool_call>\n\n<tool_call>{\n  \"arguments\": {\n    \"gene_name\": \"TERF2\", \"organism\": \"9606\", \"sort_by\": \"resolution\", \"tool_name\": \"retrieve_protein_structure_b
```

只用molbench-ms的三类问题(一共373条数据)

SFT训练完成 – 测试环境(工具调用)还没ready所以还没测

数据部分	是否计算SFT梯度	
system prompt	否	
用户问题	否	
历史observation	否	
assistant <thought>	是	
assistant <tool_call> 标签	是	
tool name	是	
tool arguments和参数值	是	
assistant本地工具调用, 如Read/Write/Edit/Bash	是	
assistant <final_answer>	是	
observation里的工具返回值	否	
user/assistant角色模板前缀	基本不监督	
assistant结束标记	参与assistant span监督	

SFT超参数		
本次实际配置:		
参数	值	意义
Model	Qwen3.5-9B	基础模型
Epoch	1	373条数据只遍历一次, 降低小数据集过拟合
ROLLOUT_BATCH_SIZE	373	一次外层数据读取覆盖完整epoch
GLOBAL_BATCH_SIZE	1	每条轨迹产生一次optimizer update
Optimizer steps	373	373条x1 epoch
Peak LR	5e-6	SFT最高学习率
Min LR	5e-7	cosine decay最终学习率
Warmup	5%	前约19步从低学习率逐渐升到峰值
LR schedule	cosine	warmup后平滑下降
Adam β_1/β_2	0.9/0.95	一阶、二阶动量平滑参数
Weight decay	0.1	抑制权重过度漂移
Gradient clipping	1.0	更新前裁剪过大的梯度
TP/PP/DP	4/2/1	4路tensor parallel、2路pipeline parallel, 无data parallel
Max tokens/GPU	16384	动态batch中单GPU token预算
Dropout	0	延续基础模型预训练/微调常用设置
Recompute	full	用额外计算换显存, 支持超长轨迹
Sequence parallel	开启	TP下分摊序列相关激活
Flash attention	开启	降低长序列attention开销
6BS=1 是因为数据只有373条且轨迹长度差异很大。它保证不需要填充或丢弃奇数尾部, 同时每条完整轨迹都产生一次更新。缺点是梯度方差较大, 因此采用了较小学习率、单epoch和gradient clipping。		

ToolRL:
数据:从每个tool_call处截断保留历史消息,得到3028条ToolRL step

对于每个固定history state, 模型生成:

```
xml
<thought>...</thought>
<tool_call>{"tool_name":"...", "arguments":{"..."}}</tool_call>
```

或者:

```
xml
<thought>...</thought>
<final_answer>{...}</final_answer>
```

数据部分	是否计算ToolRL梯度
固定system/user/history	否
历史assistant thought/tool call	否
历史observation	否
当前模型新生成的thought	是
当前模型新生成的tool call	是
当前模型新生成的final answer	是
teacher target文本	不直接计算交叉熵
teacher tool/final结构	只用于计算reward
真实MCP observation	不存在, formal ToolRL不调用MCP

6. 当前使用的ToolRL reward

本次使用:

```
text
TOOLRL_REWARD_MODE=official
```

实现位于 [molclaw_reward.py \(line 123\)](#) .

总reward:

$$R = R_{\text{format}} + R_{\text{correctness}}$$

参数	值	意义
Dataset	3028 steps	固定history-only decision states
NUM_ROLLOUT	757	3028 / 4, 完整遍历一次
ROLLOUT_BATCH_SIZE	4	每轮处理4个不同decision state
N_SAMPLES_PER_PROMPT	4	每个state采样4个候选
Responses/rollout	16	4 prompts x 4 candidates
GLOBAL_BATCH_SIZE	16	每轮16个response产生一个optimizer step
Optimizer steps	757	每个rollout一个step
Temperature	1.0	保证同一prompt生成有一定差异, 供GRPO排序
Response max length	4096	当前decision最大生成长度
Prompt max length	98304	支持很长的历史轨迹
Context max length	102400	prompt与response合计上限
LR	2e-7	比SFT低25倍, 避免RL破坏已有能力
LR schedule	constant	整轮使用固定小学习率
PPO clip low/high	0.2/0.28	probability ratio约束区间约为 [0.8, 1.28]
Entropy coefficient	0	不额外奖励随机性

Report 2026.08.05

H集群 GPU worker不能访问网络, 所以不能连到

`MOLCLAW_SCP_SERVER_URL=http://180.184.86.2:32208/mcp`

解决方案: no GPU worker 连 mcp server, 本地ip当跳板

一、这次 MolClaw 的真实实现路径

1. GPU worker 里的业务进程仍然访问真实目标地址: `http://180.184.86.2:32208/mcp`。
2. GPU worker 不能直连外网目标, 所以通过环境变量把流量先导向 no-gpu: `HTTP_PROXY=http://<no-gpu-ip>:13208`。
3. no-gpu 上跑一个“纯转发 relay” (监听 `13208`), 它不理解 MCP 协议, 只做字节流转发。
4. relay 把流量继续转发到集群 HTTP 代理: `httpproxy-headless...:3128`。
5. 集群 HTTP 代理再访问最终 MCP 服务 `180.184.86.2:32208`, 响应按原路返回。

一句话:

`GPU进程 → no-gpu:13208 → 集群HTTP代理:3128 → MolClaw MCP`

- 81 个实时 MolClaw MCP 工具
- 6 个本地沙箱工具: `Read / Write / Edit / Bash / Grep / Glob`

Qwen3.5-9B

使用 9B ToolRL checkpoint `iter_0000756` , 串行 `MAX_WORKERS=1` 、温度 0、 `MAX_STEPS=0` , 五题全部完成:

测试	结果	严格协议
EGFR retrieval → <code>fix_pdb</code>	成功, 包含失败后重规划	4/4 decision 合法
<code>is_valid_smiles("CCO")</code>	成功	2/2 合法
<code>visualize_molecule("CCO")</code>	成功	2/2 合法
EGFR retrieval → <code>visualize_protein</code>	成功, 包含失败后重规划	4/4 合法
Write <code>"smoke ok"</code> → Read	成功	3/3 合法

1. EGFR retrieval → `fix_pdb`

Retrieve a human EGFR protein structure by gene name, then repair the retrieved PDB with `fix_pdb` using `remove_water=true`, `remove_heterogens=true`, and `add_hydrogens=false`. Return the repaired canonical artifact in an e2e `final_answer`. Use only the two necessary MolClaw calls.

2. `is_valid_smiles("CCO")`

Call `is_valid_smiles` exactly once for the SMILES CCO. After receiving the observation, return a concise e2e `final_answer` reporting whether it is valid. Do not call any other tool.

3. `visualize_molecule("CCO")`

Call `visualize_molecule` exactly once for the SMILES CCO. After receiving the observation, return a concise e2e `final_answer` containing the canonical image artifact. Do not call any other tool.

4. EGFR retrieval → `visualize_protein`

Retrieve a human EGFR protein structure by gene name, then call `visualize_protein` on the retrieved canonical PDB artifact. Return the canonical visualization

```

{
  "id": "probe_frequent_retrieve_fix_egfr",
  "task_id": "probe_frequent_retrieve_fix_egfr",
  "task_type": "e2e",
  "data_source": "manual_prompt",
  "evaluation": true,
  "tool_catalog_sha256": "2ccbfef38e685339deb2d8dde1bd2f1aa6297744ff1c8fb5eca113f5d131262",
  "workspace": "<artifact:local/>",
  "max_steps": 0,
  "rollout_mode": "canonical_react_strict",
  "parse_recovery_enabled": false,
  "allow_parse_recovery_override": false,
  "actions": [{
    "step": 0,
    "raw_response": "<thought>I need to retrieve a human EGFR protein structure by gene name, then repair it with fix_pdb. Let me start by
retrieving the structure.</thought>\n<thought>I'll retrieve the human EGFR protein structure by gene name and then repair it with
fix_pdb.</thought>\n<tool_cal>{\"arguments\":{\"gene_name\":\"EGFR\",\"organism\":\"Homo
sapiens\",\"sort_by\":\"resolution\"},\"tool_name\":\"retrieve_protein_structure_by_gene_name\"}</tool_call><|im_end|>",
    "finish_type": "stop",
    "parsed": {
      "ok": true,
      "decision_type": "tool_call",
      "thoughts": ["I need to retrieve a human EGFR protein structure by gene name, then repair it with fix_pdb. Let me start by retrieving the
structure.", "I'll retrieve the human EGFR protein structure by gene name and then repair it with fix_pdb."],
      "tool_calls": [{
        "tool_name": "retrieve_protein_structure_by_gene_name",
        "arguments": {
          "gene_name": "EGFR",
          "organism": "Homo sapiens",
          "sort_by": "resolution"
        }
      }],
      "raw_payload": {
        "arguments": {

```

1. 工具参数语义不够稳定

模型调用：

```
json
{
  "gene_name": "EGFR",
  "organism": "Homo sapiens",
  "sort_by": "resolution"
}
```

但该工具要求 `organism` 使用 taxonomy ID，例如 `"9606"`。这导致真实 API 400。

说明模型认识工具名和大致用途，但未必严格遵守参数语义。

是因为organism应该是有默认值(表示人类)，训练数据里直接没有organism而现在mcp tool 要求输入organism，所以智能体会直接自然语言填充

3. 复杂指令下容易过度生成

此前 Skill→Write→Read 任务中，模型试图写很长的 `run_log.md`，导致 tool-call JSON 在 4096 token 限制处被截断。

表现为：

text

任务本来只需写一句话

- 模型生成完整工程报告
- JSON 未闭合
- 工具无法执行

原子化成“只写 `smoke ok`”后则完全成功。

4. Reasoning 存在重复和冗长

成功轨迹中经常连续生成多个意思相近的 `<thought>`，final 前也会多次总结。这会：

- 浪费生成 token；
- 增大 JSON 截断概率；
- 拉长工具交互时间；
- 在长任务中挤压上下文。

在LLM clean步骤让其清洗临近的语义相近的thought

5. 多步骤组合的协议稳定性不足

单工具及简单两步任务表现不错，但任务同时包含 Skill、文件操作、报告生成和 final 时，协议稳定性明显下降。

因此当前比较准确的判断是：

模型已经学会 XML ReAct、真实工具调用、observation 后继续决策、artifact 传递以及简单失败重规划，但复杂多步骤任务中的参数精确性、输出克制和协议稳定性仍不够成熟。

正面结果 [添加到对话](#)

- `visualize_protein` 在训练集中为 0 次，模型仍能根据实时 schema 正确调用。
- `visualize_molecule("CCO")` 也属于未见过的精确调用。
- 最小 Write→Read 能正确操作 workspace。
- 普通工具失败不会导致整题终止，模型可以 replanning。

不过现在只有 5 道人工 smoke，尚不能评价 MolBench 科学任务正确率。

完整跑过molbench-ms-1上的两个问题,

acc = 1.0

precision = 1.0

recall = 1.0

f1 = 1.0

sensitivity = 1.0

specificity = 1.0

validity = 1.0

n_samples = 2

是否在项目训练集中见过原题或答案

我检查了训练该 checkpoint 使用的原始 373 条 canonical 数据:

[📄 373 条训练数据](#)

结果:

- 两个完整问题均无精确匹配;
- 第一题的 10 个候选 SMILES 在全部训练 messages 中命中 0 个;
- 第二题的 10 个候选 SMILES 也命中 0 个;
- 两个最终答案 SMILES 均未出现在训练 messages 中。

`overlap_audit.jsonl` 中确实有 4 条训练重叠, 但它们属于 MS-2, 被 adapter 排除, 并不是这里评测的两条 MS-1。

因此, 就本项目可查的数据而言, 没有发现训练集答案泄漏。

最终判断

可以得出:

这个智能体在这两道 MS-1 题上, 真实调用了合适的计算工具, 并根据真实 observations 正确应用了全部筛选条件, 最终答案完全正确。

但不应扩大为:

智能体已经完全掌握 MS-1。

完整molbench测评因为程序问题结果没保存, 正在重跑

在8卡worker上能训练：

Qwen3.5-27B(4卡也能训)

Qwen3.5-122B-A10B-FP8(LoRA rank=32)

- `linear_qkv`
- `linear_proj`
- `shared_experts.linear_fc1`
- `shared_experts.linear_fc2`

没有对 GDN/Mamba 模块、Embedding、LM Head、LayerNorm、Router 等全部层做 LoRA。

从tool-kg采样的数据：这一批预备采样240条，还在运行，主要因为有很多耗时很长的任务

看一下问题

Data-Pipe 用时统计

统计口径是正式旧 100 加当前已完成的 39 条，共 139 个严格成功样本；没有把重复 partial run 和 smoke 重复计入。

指标	全部 139 条	当前新批次 39 条
最短	14 秒	6.2 分钟
P25	6.3 分钟	17.4 分钟
中位数	18.1 分钟	28.1 分钟
平均值	33.2 分钟	39.8 分钟
P75	38.0 分钟	42.4 分钟
P90	73.3 分钟	87.8 分钟
P95	121.6 分钟	127.5 分钟
最长	9.41 小时	2.93 小时

当前新批次更能代表现在的题目和运行环境。其耗时分布为：

- 23.1%：不超过 15 分钟
- 33.3%：15–30 分钟
- 25.6%：30–60 分钟
- 10.3%：1–2 小时
- 7.7%：2–4 小时

Report 2026.08.12

373条 molbench数据上训练9B模型 - test on molbench

严格口径结果

Suite	题数	合法 final	主指标成功	指标
MS-1 property filtering	50	40	36	Accuracy 72.00%
MS-2 activity comparison	33	11	5	Accuracy 15.15%
MO edit	39	0	0	Correct rate 0%
MO optimization	39	0	0	Success rate 0%
合计	161	51	41	严格成功率 25.47%

Suite	数量	轨迹中的主要现象
MO edit	14	反复讨论如何修改 SMILES、不断说“换一种方法”，但不真正提交 final
MS-1	6	4 题反复念叨 <code>selected_smiles must be a list</code> ；2 题已经得到答案但不断重复结论
MS-2	15	7 题退化为大量 <code>QKQK...</code> / <code>QVQV...</code> token；6 题重复同一计划；2 题已有完整比较结论但不提交 final

根因	数量	典型现象
错误 final schema 无限重试	5	反复使用 <code>modified_smiles</code> ，而协议要求 <code>output_smiles</code>
相同失败工具无限重试	6	参数、工具和错误几乎不变
成功工具无限重复	6	工具已经成功，模型仍不断重新计算或验证
MCP 首步连接失败	6	第一轮调用即因连接超时退出

问题:

- 1.没有学习到read skill的行动 – ToolRL中Read、Write等本地工具没有被视作decision
- 2.长 tool-call 被token限制截断 – `server\_file\_to\_base64`等工具的输入/输出太长
- 3.常见临近重复语义 thought, 偶见循环重复导致失败

第二次9B模型的训练：

365条 molbench问题轨迹(剔除8条和molbench存在相同实体的)
+145条 tool-kg采样问题的轨迹 == 510条样本

出现严重的“模型循环行为”，跑了12个molbench题目

其中有两类非常典型的循环：

- 成功后仍重复：例如同一调用连续成功 82、84、60 次，却始终不生成 final answer。
- 失败后机械重试：部分样本在得到相同错误后连续重复 384、449、493 次。

这次训练还把GRPO替换成了REINFORCE++

– 后者从baseline计算reward，可能导致过拟合到数据集的分布了

且没有有效修复“把本地工具也跟molclaw工具一致地视作decision”

按照qwen3.5-9B模型文件中记录的：

```
text_config.max_position_embeddings = 262,144
tokenizer.model_max_length = 262,144
```

按照这个上限，选用：

```
max context  = 262,144
max response = 16,384
max prompt   = 245,760
```

其中server_file_to_base64工具超长的
暂时通过在推理框架里添加了用编码替换
base64字符串的方式能解决推理时的超长
问题

这里核心原因还是上下文导致的prompt超长
(主要是前序的长tool-call和tool
observation)

被排除的工具 decision

以下“decision 数”表示多少个排除 decision 包含该工具；多调用 decision 可能同时计入多个工具。“调用数”则统计 decision 内的实际调用次数。

工具	Decision 数	实际调用数
molecule_docking_quickvina_fullprocess	65	104
server_file_to_base64	53	61
Write	35	35
Read	26	26
Bash	19	19
base64_to_server_file	11	11
interaction_visualizer	5	6
Edit	3	3
extract_and_save_chains	3	3
fix_pdb	2	2
pred_binding_affinity_boltz2	2	2
analyze_protein_ligand_interactions	2	2
foldx_tool	2	2
calculate_mol_basic_info	1	1
convert_smiles_to_format	1	1
is_valid_smiles	1	1
extract_pdb_chains	1	1
calculate_pdb_composition_info	1	1
pred_pocket_prank	1	1

这些超长项的原因可以归纳为：

原因	影响
用 <code>Read</code> 读取 PNG，返回数十万字符	最大的 prompt 膨胀来源
多轮批量 docking/Boltz2，重复 SMILES、蛋白序列和结果	docking 等后期 decision 超长
teacher 在简单工具调用前生成几十万字符 thought	<code>Read/Write/Bash/server_file_to_base64/extract_chains</code> target 超长
base64 直接内联进工具参数	<code>base64_to_server_file</code> 直接 target 超长
完整报告内联到 <code>Write.content</code>	部分 Write target 超长
一次传入大型 SMILES 列表	<code>is_valid_smiles</code> target 超长
最终答案前反复汇总和校验	final target 超长

现在数据集规模:

$365 + 240 = 605$ 条数据

check 240 条数据的质量:

- 轨迹和预期tool-chain做一致性分析
- 对轨迹的tool-chain做diversity分析

- Expected recall: 预期工具执行了多少?
- Actual precision: 实际工具中有多少属于预期?
- Pairwise Jaccard distance: 不同轨迹之间使用的工具集合有多不一样?

Average expected recall = 0.7649 平均每条轨迹覆盖了约 **76.5%** 的预期工具

Average actual precision = 0.3919 轨迹使用的不同工具中, 平均只有 约 **39.2%** 出现在 Tool-KG 预期链里

Pairwise tool-set Jaccard distance = 0.8119 任意两条轨迹的工具集合平均只有 约 **18.8%** 的 union 是共同工具

240 题的主模式统计

类别	主模式	数量	占比
B	已知配体比较 / screening / ranking	80	33.3%
A	单复合物 binding-mode / interaction / hotspot 机制分析	51	21.2%
C	小分子生成与 lead optimization	38	15.8%
D	Peptide / protein binder 生成与设计	25	10.4%
E	蛋白结构表征 / preparation / stability / pocket	14	5.8%
F	分子 property / descriptor / QC / format / similarity	13	5.4%
G	MD / free-energy / protein-protein / dynamics workflow	10	4.2%
H	Disease-reversal / therapeutic prioritization	9	3.8%
总计		240	100%

1. RL:reward的细度可能不需要精确到参数跟teacher traj 一致
2. 针对一致重复tool-call的情况可以提高final的reward占比
3. 对超长的样本可以有一个压缩机制(训练的时候可能只需要一个抽象的上下文)
4. 有意识地区分开训练模型“拿到问题的planning”和“每一个步骤的tool call”的能力()
5. check 采样问题的工具链和molbench对应类别问题的一致性/区分度

Report 2026.08.19

1. check 采样问题的工具链和molbench对应类别问题的一致性/区分度

1. Tool-KG 与对应 MolBench 类型

对比	全 cross similarity	最近邻 similarity	词表 Jaccard	调用分布 JS similarity
F ↔ MS1/PF	0.1195	0.3181	0.4375	0.3820
B ↔ MS2/AC	0.1603	0.4531	0.6531	0.6528
B ↔ MS3/VS	0.2038	0.4625	0.6087	0.3278

主要解释：

- 三组概念对应类型的逐轨迹工具集合一致性都不高。
- B 与 MS2/AC 最接近，尤其调用频率分布相似度达到 0.6528。
- B 覆盖了 MS3/VS 使用的全部 28 个工具，但 VS 中 docking 被大量重复调用，因此调用分布相似度只有 0.3278。
- MS1/PF 的全部 7 个工具都包含在 Tool-KG F 的 16-tool 词表中，但 F 的任务和执行路径更宽，所以全 cross similarity 仅 0.1195。

2. MolBench MS1/2/3 内部

类型	组内一致性	Diversity distance	Tool-set 唯一率
MS1/PF	0.7387	0.2613	0.0828
MS2/AC	0.3071	0.6929	0.7788
MS3/VS	0.6011	0.3989	0.7449

结论：

- MS1/PF 高度模板化：一致性最高、多样性最低。
- MS2/AC 内部最分散，多样性最高。
- MS3/VS 工具集合较一致，但工具顺序和具体组合仍有明显变化。
- 三类之间，MS2/AC 与 MS3/VS 最接近，平均 similarity 为 0.2448。

3. Tool-KG A-H 内部与类别间

类别	组内一致性	Diversity distance
A	0.3452	0.6548
B	0.2466	0.7534
C	0.1705	0.8295
D	0.1957	0.8043
E	0.2679	0.7321
F	0.3381	0.6619
G	0.2656	0.7344
H	0.3289	0.6711

总体上，A-H 每个大类内部都保持了较高 diversity：

- A 相对最一致，但 similarity 也只有 0.3452。
- C 的 diversity 最高，distance 为 0.8295。
- A-H 中最接近的两个类别是 A 与 E，但 similarity 也只有 0.2591，说明各主类别之间具有较明显的工具链区分。

240条采样问题样本 + 365条molbench样本

模型反复生成了类似:

XML

```
<tool_call>
{"tool_name":"tool_a","arguments":{...}},
{"tool_name":"tool_b","arguments":{...}},
{"tool_name":"tool_c","arguments":{...}}
</tool_call>
```

ToolRL的官方训练集的格式为:

```
<tool_call>
{"tool_name":"tool_a","arguments":{...}}
{"tool_name":"tool_b","arguments":{...}}
{"tool_name":"tool_c","arguments":{...}}
</tool_call>
```

GRPO 4 个 rollout 全都生成同一种错误格式 → reward 都是 -0.5 → 不产生梯度

而我们的训练集里严格要求里面只能有一个 <tool_call>

```
{"tool_name":"tool_a","arguments":{...}}</tool_call>
```

基础模型强烈的 JSON 先验又倾向在多个对象之间插入逗号

之前训练:把每一次tool-call看作一个decision单元

ToolRL 官方 4K RL 数据本质上是 step-level 数据。一个样本 target 大致是:

XML

```
<think>... </think>
<tool_call>
{"name":"tool_a","parameters":{...}}
{"name":"tool_b","parameters":{...}}
</tool_call>
```

而原本的toolrl是把“连续的tool-call和其前面的think”看作一个decision单元

所以对训练的数据集也做了调整成这样的格式

既然qwen3.5的基础模型先验为带逗号的,那可以用带逗号的

现在训练样本格式：

```
{
  "schema_version": "drug_agent_sft_toolrl_turn_v1",
  "id": "react_pf_303eea476077228e",
  "messages": [{
    "role": "system",
    "content": "You are a scientific agent operating under the canonical ReAct protocol.\nUse only recorded MolClaw calls and supported local file calls from the recorded execution.\nWrite scientific reasoning inside\n<thought>...</thought>, calls inside <tool_call>...</tool_call>, recorded results inside\n<observation>...</observation>, and the grounded result inside\n<final_answer>...</final_answer>.\nNever invent a tool result. Observations must come from execution history, and the final answer\nmust be supported by the task result and recorded observations.\nWithin each reasoning/action segment, put all tool calls inside exactly one <tool_call>...</tool_call> container as newline-separated JSON objects without commas or a JSON array.",
    "step_loss_mask": 0
  }, {
    "role": "user",
    "content": "You are a cheminformatics assistant.\n\nTask:\nFrom the following SMILES list, output ALL molecules that satisfy ALL constraints below.\nYou must ONLY choose from the given SMILES and output them EXACTLY as provided (original strings).\nIf none satisfy, output an empty line.\n\nSMILES:\nClC1CCCC(Cl)C1N=C1NCCCN1\nCCN(CC)CCNC(C(=O)OCCC(C)C)c1cccc1\nCCc1nc(N)nc(N)c1-c1ccc(Cl)cc1\nOc1ccc(Cl)cc1Cc1cc(Cl)ccc1O\nCCN(CC)CCNC(=O)c1c(C)[nH]c(/C=C2\\C(=O)Nc3ccc(F)cc32)c1C\nCN(C)CCCN1c2ccccc2CCc2ccccc21\nCOC1=CC(=O)C[C@@H](C)[C@]12Oc1c(Cl)c(OC)cc(OC)c1C2=O\nCOc1cc(Cc2cnc(N)nc2N)cc(OC)c1OC\nCCN(CC)C(C)CN1c2ccccc2Sc2ccccc21\nCN(C)CCC=C1c
```


官方 MolBench 分数以全部任务为分母，非 final 计零：

Suite	Accuracy	其他指标
MS1	38.0%	Precision 44.48%, Recall 48.83%, F1 45.09%, SMILES validity 51.0%
MS2	3.03%	Valid-choice rate 6.06%

MS1 的 26 个 final 中有 19 个 exact correct，条件正确率约 73.1%；MS2 的两个 final 只有一个正确。

兼容修复确实生效：

- strict-original-format rate: 93.68%
- compat-assisted: 40 个 action、8 个任务
- 这 8 个任务全部是 MS2，已知 comma-separated Extra data 格式成功执行了工具
- 但 8 个任务最终都继续循环并触发 length

因此已知 serialization bug 不是 v4 的主要剩余瓶颈。真正问题是：

1. 轨迹严重不收敛。44/83 触发长度限制；MS2 action 均值 25.8、最大 99。
2. 工具选择幻觉。61 次未知工具调用，其中 calculate_mol_admet 60 次、pred_admet 1 次；tool-name validity 96.13%。
3. 工具链执行不稳定或选择不合理。共 199 次工具执行失败，MS2 占 164 次；大量失败来自蛋白结构检索、docking、文件转换和 Karmadock 链路。另有 26 次 MCP transport failure，但同时有 1317 次成功执行，因此不是 relay 整体掉线。
4. MS2 策略明显不对：27 个 length、3 个 parser failure、1 个 timeout，只产生两个 final。
5. 剩余 10 个 parser failure 不是已知 comma-separated bug：
 - 7 个 final answer 的 evidence 不是规定的 list
 - 3 个确实是 malformed JSON
6. 最终答案仍有 <artifact:local/...> 污染 SMILES，拉低 validity。
7. Arguments schema accuracy 为 100%，说明主要问题不是参数 JSON 结构，而是工具名、任务规划、工具结果利用和终止决策。

1. 参数不必完全匹配 teacher trajectory

状态：已落地。

当前 production 使用 hierarchical gated reward:

纯文本

格式

- 工具集合
- required arguments
- critical arguments
- configurable arguments

对应的大致 reward 层级是：

情况	Reward
非法格式	-0.5
完全错误工具	-0.4
部分工具集合正确	约 -0.05~0.30
工具正确但缺 required argument	0.30~0.50
identity-critical 参数错误	0.55~0.70
configurable 参数 schema/range 非法	0.72~0.85
合法的替代配置	0.90
teacher-equivalent	1.00

参数分成：

- identity-critical: SMILES、protein、target、ID、path、artifact、chain 等，仍要求严格；
- configurable: `top_k`、`top_n`、采样数和可调阈值等，只要求 schema、类型和范围合法，不要求 teacher exact。

因此“选对工具”已经成为主要门槛，参数是在通过工具门槛之后再分级奖励，而不是简单的固定加权。

对一些如 `top_k`, `top_n` 等的填数的参数不要求完全匹配

2. 重复 tool call 时提高 final 占比

状态：已落地，但不是通过复制 final。

我们把普通 `repeated_tool_call` 收缩成更可靠的 `no_progress_repeat`：

纯文本



完全相同的规范化调用

+

上一次已经得到可用成功结果

+

中间没有新的成功状态变化

满足时才删除这个重复 decision。

以下 retry 不会被删除：

- 上一次 timeout；
- 上一次 error；
- 没有 observation；
- 中间状态发生变化后重新调用。

production 物化中：

- full 删除 18 个 no-progress repeat；
- mol 删除 4 个。

final 每条轨迹正常保留一次，不复制、不乘额外 reward。由于无进展的中间 step 被删除，final 在对应轨迹中的相对训练占比自然提高。

这是对原意见的直接实现：

纯文本



减少无进展 action 的 credit

→ final 相对占比上升

不需要每个decision单元都参与训练：
连续的相同状态的tool-call只保留一个

3. 超长样本压缩

状态：压缩机制已落地；当前 v6-final 实际走的是确定性路径，没有调用 LLM。

硬上限已经固定为：

纯文本



```
max context   = 262,144
max prompt    = 245,760
max response  = 16,384
summary       ≤ 32,768
```

materializer 的顺序是：

1. 使用真实 Qwen chat template 精确计数。
2. typed observation microcompact：
 - blob/base64/文件内容 → 类型、长度、hash、head/tail、artifact 引用；
 - 大数组/JSON → schema、count、关键字段；
 - error log → status、error type、核心错误。
3. 保留最近完整历史。
4. 仍超限时，对移除的早期历史生成结构化 summary。
5. target 超过 16,384 时直接排除，禁止截断 label。

当前 full production：

- 68 个 prompt 被压缩；
- 最大最终 prompt 245,621；
- 最大 target 15,279；
- 全部符合三项上限。

mol production 的最长 prompt 只有 89,518，因此没有样本需要压缩。

虽然 Claude summarizer skill 和缓存逻辑已经实现，但当前 v6-final manifest 是：

纯文本



```
semantic_summarizer = none
```

也就是说，这批正式数据的 68 个压缩样本使用的是 deterministic structured summary，并没有付费调用 LLM。以后确定性摘要无法保证语义时，可以离线重新物化并启用 Claude，而不需要修改 converter 或训练器。

4. 区分 planning 与逐步 tool call

状态：SFT 数据层已落地，RL 独立优化暂缓。

v5 LLM clean 已对全部轨迹处理首位 planning：

- full 605 条：
 - prepend planning：339；
 - rewrite first thought：266。
- mol 365 条：
 - prepend：303；
 - rewrite：62。

planning annotation sidecar 也已保留。

当前训练仍然是：

纯文本



SFT → ToolRL

没有独立 Plan-SFT。

在 SFT 中，首位高层 planning thought 会受到 teacher supervision；但在 ToolRL 中：

- planning 不成为独立 decision；
- 没有 planning reward；
- 不复制 planning 样本；
- 不把 first thought 标成特殊 role；
- thought 和随后的 tool calls 一起构成 action；
- rollout thought token 只通过整个 action 的 sequence advantage 间接更新；
- teacher thought 的语义不进入 ToolRL reward。

所以这一项目前是有意识地“完成数据准备，但暂不单独做 planning RL”，符合后来确定的范围。

